

CS 1910

Computer Science I  
Lecture 2  
Computational Thinking

# ANOTHER “IS THIS CS”?

- Is this CS?



- Physical environment mapping
- Stitching of images together to create a single virtual environment
- Rendering of the stars based on the current night sky.

<https://www.stonehengeskyscape.co.uk/>



## PAIR TASK

5 minutes

- Find a partner
- One partner will **communicate**, one will **illustrate**
- The communicator receives a picture
  - Do **NOT** show your partner the picture
- The illustrator gets out a pencil and blank paper
  - Do **NOT** show your partner your paper or work
- Using only verbal instructions the communicator should tell the illustrator what to draw
  - No partner should peek at the others work and only the communicator can talk, the illustrator cannot

# COMPUTATIONAL THINKING

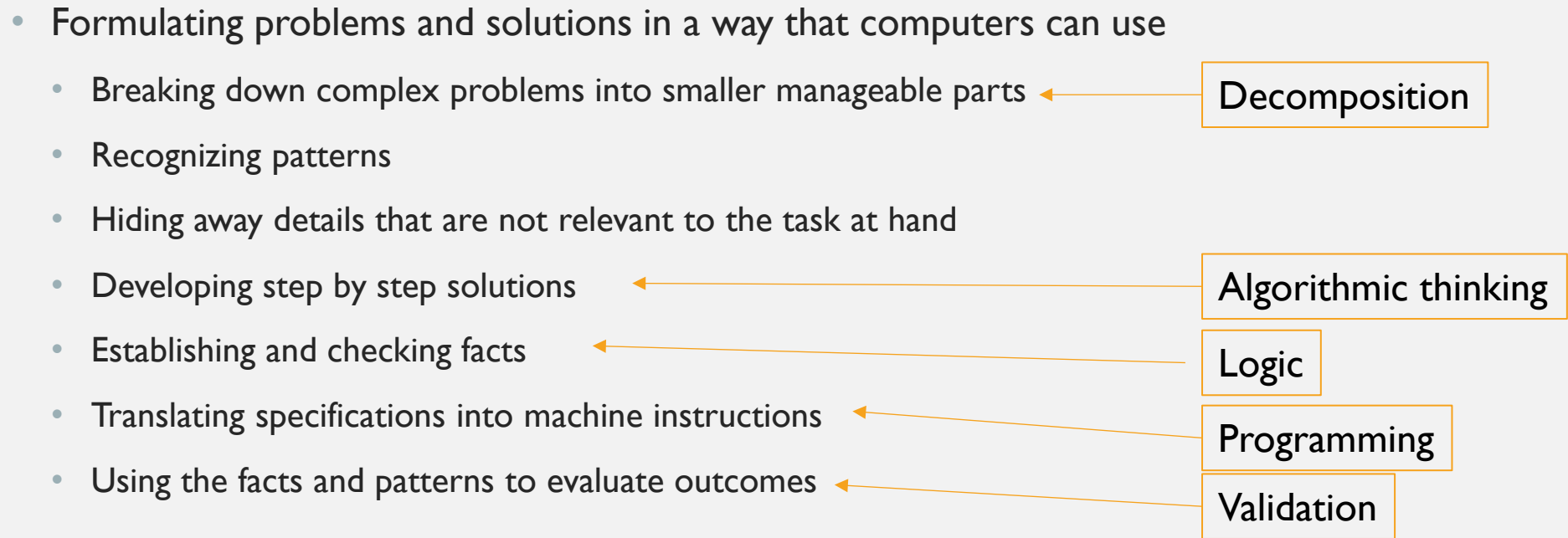
- Formulating problems and solutions in a way that computers can use
  - Breaking down complex problems into smaller manageable parts
  - Recognizing patterns
  - Hiding away details that are not relevant to the task at hand
  - Developing step by step solutions
  - Establishing and checking facts
  - Translating specifications into machine instructions
  - Using the facts and patterns to evaluate outcomes

# COMPUTATIONAL THINKING

- Formulating problems and solutions in a way that computers can use

- Breaking down complex problems into smaller manageable parts

Decomposition



- Recognizing patterns

- Hiding away details that are not relevant to the task at hand

- Developing step by step solutions

Algorithmic thinking

- Establishing and checking facts

Logic

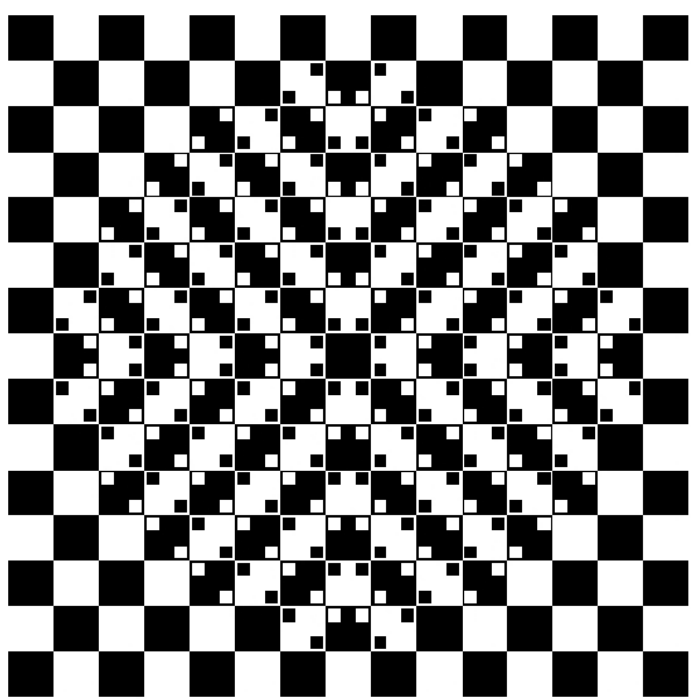
- Translating specifications into machine instructions

Programming

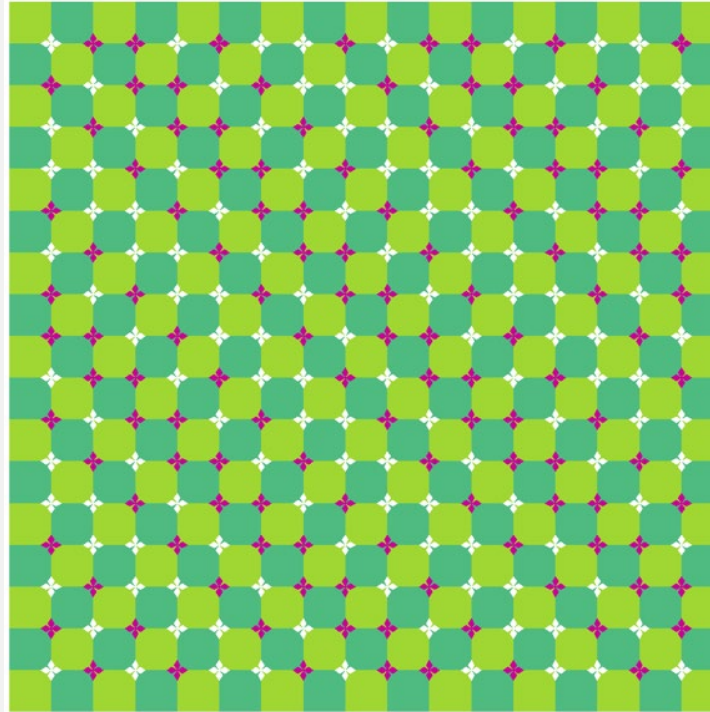
- Using the facts and patterns to evaluate outcomes

Validation

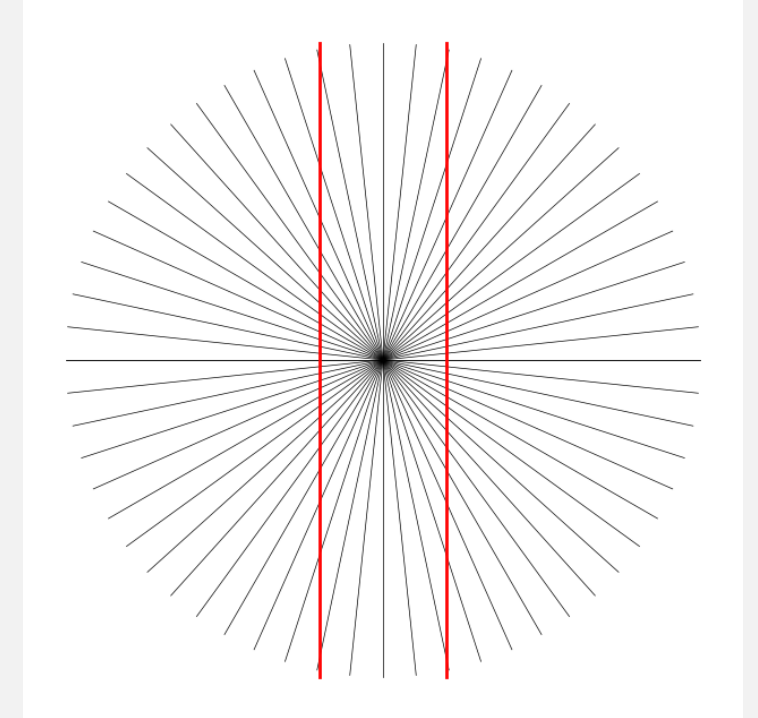
# OPTICAL ILLUSIONS



Grid pattern appears to  
bulge out of the page



Grid pattern appears to  
have waves



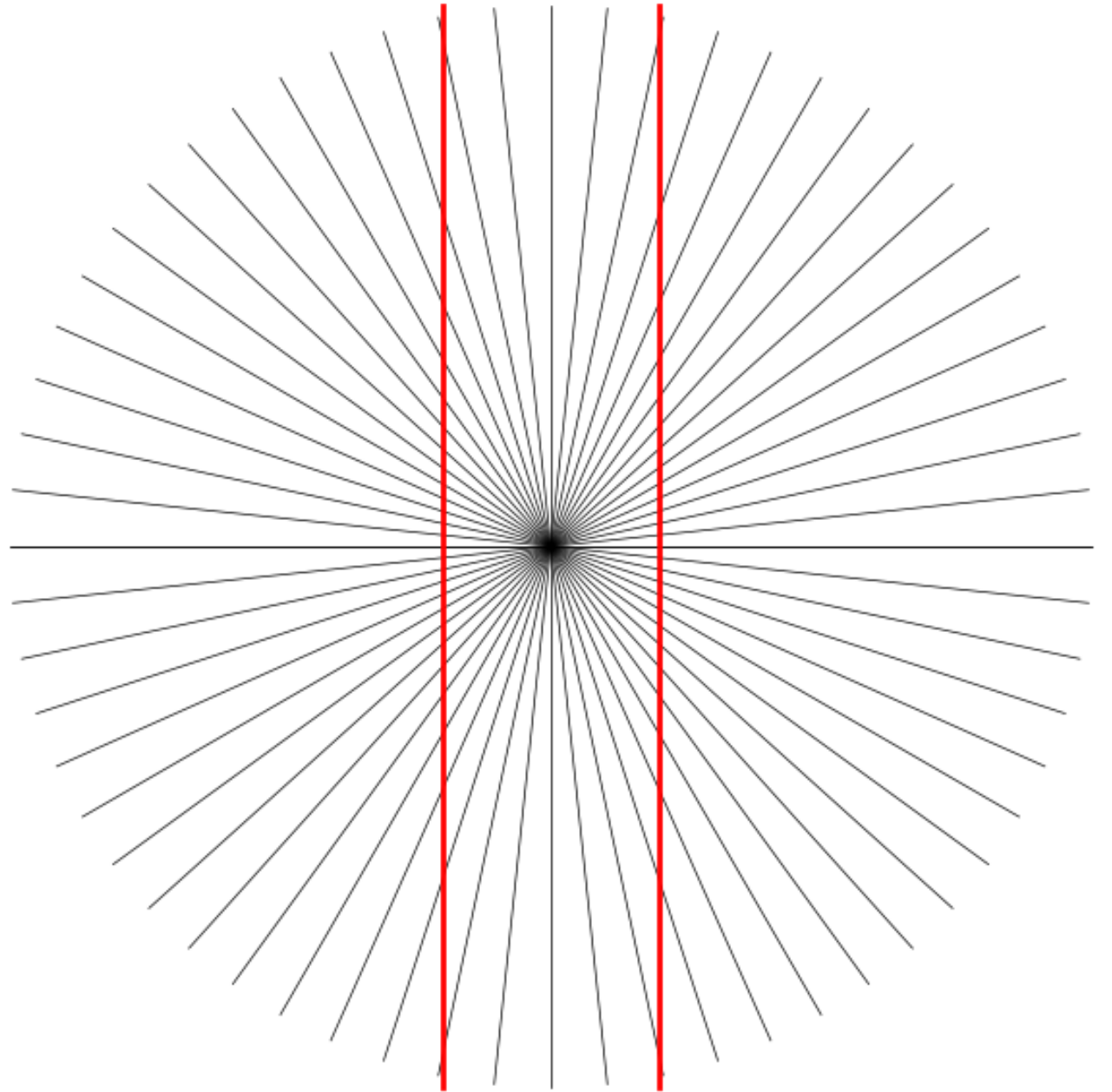
Parallel lines appear to  
curve

Suppose we want to recreate these illusions  
Can we use computational thinking to help?

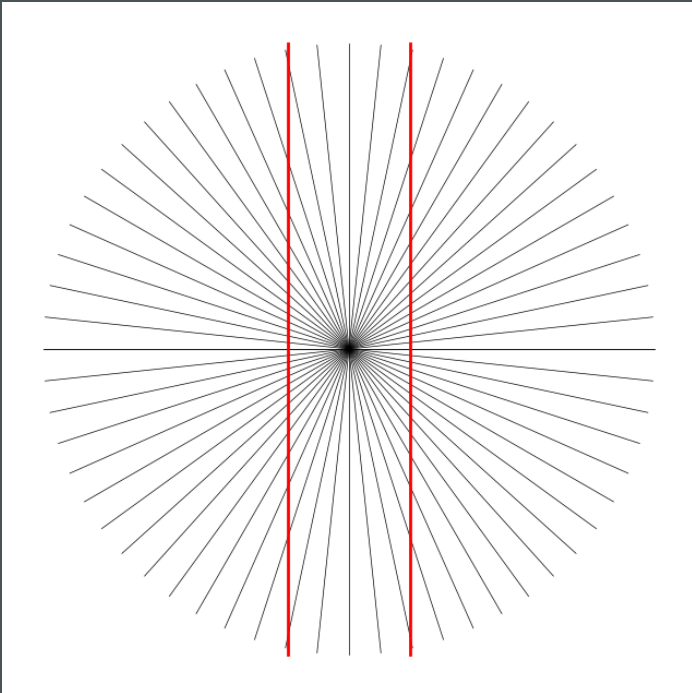
\*credit: <https://www.ritsumeai.ac.jp/~akitaoka/index-e.html>

# OPTICAL ILLUSION

- The red lines are parallel but they have the illusion of being curved



# INITIAL CONCERNS



---

## **Where do I start drawing?**

Should I begin at the center or the outer edge?

---

## **How do I make the lines straight?**

Do I need a ruler or a grid?

---

## **How do I space the lines evenly?**

Should I measure angles or distances?

---

## **What about the circle?**

How do I make it perfectly round?

---

**How do I draw the red vertical lines?** How far apart should they be?

---

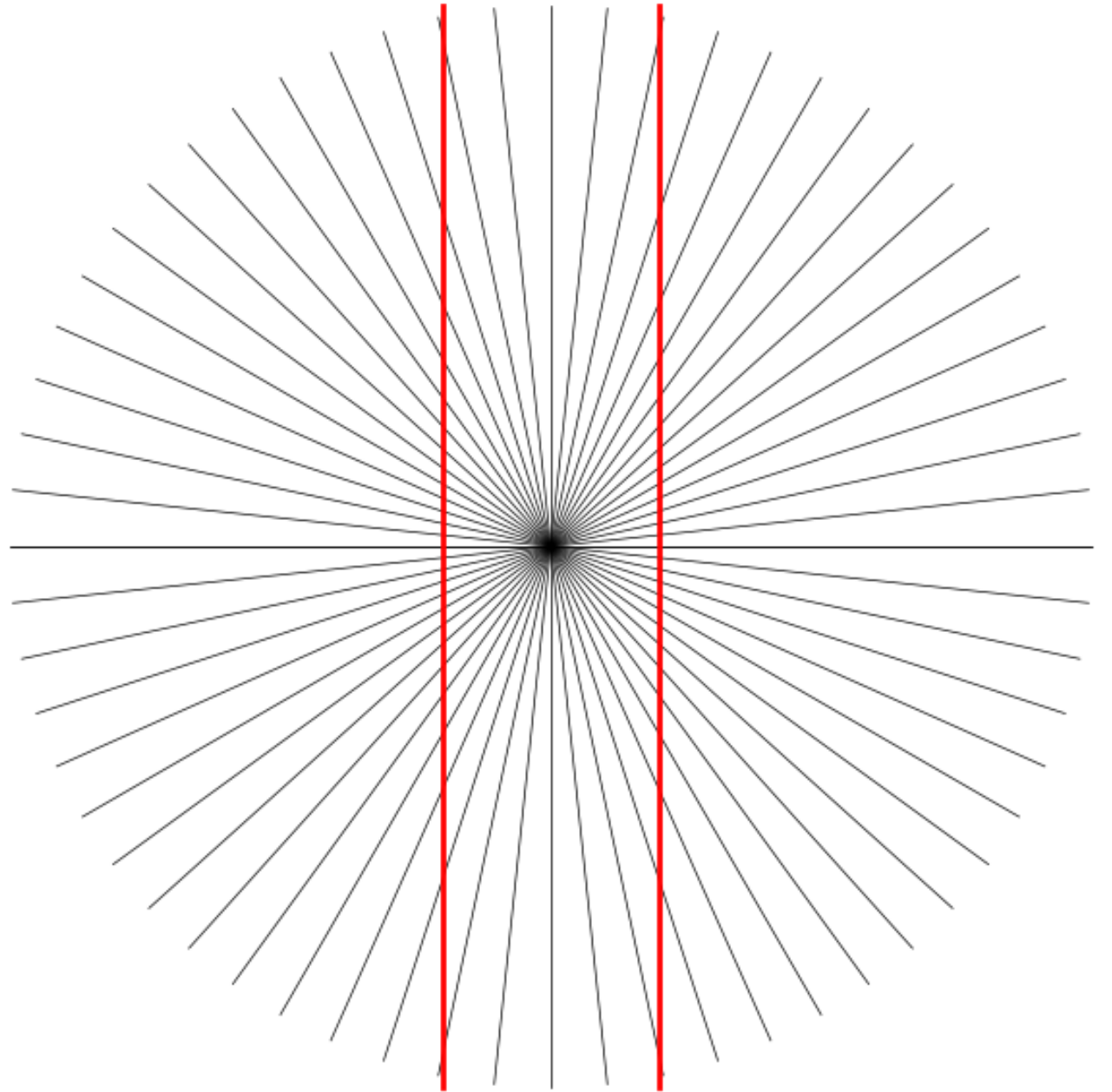
## **How do I keep everything aligned?**

Do I need to check symmetry after every line?

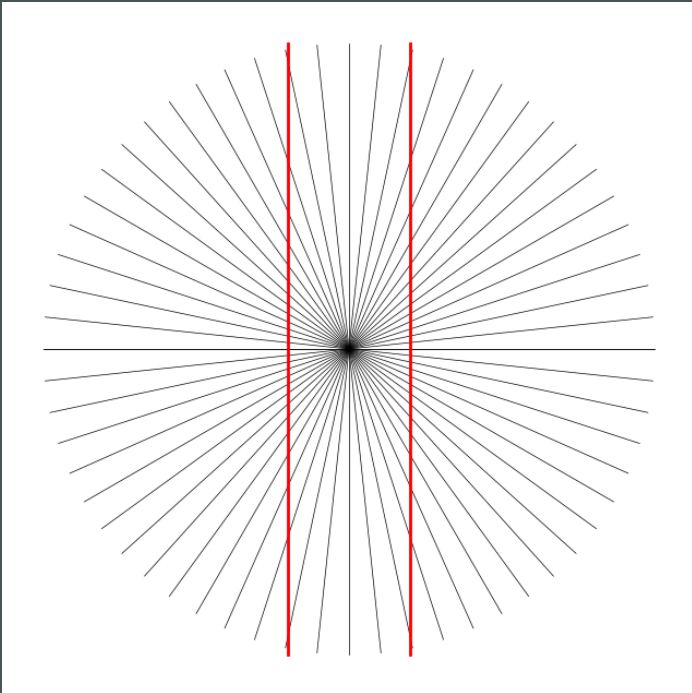


# OPTICAL ILLUSION

- Based on what you explained to your partner – what are the key features of this particular illusion?



ABSTRACTION:  
SIMPLIFY THE  
PROBLEM



---

**Center Point**

---

**Radial Lines**

---

**Circle Boundary**

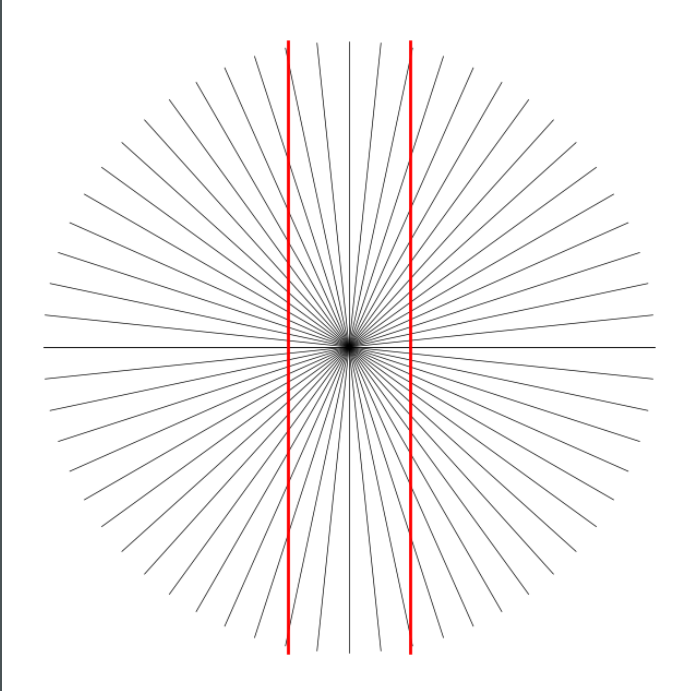
---

**Two Parallel Lines**

---

# DECOMPOSITION

Breaking the problem into manageable bits



---

**Center Point:** Define a centre point.

---

**Radial Lines:** Draw a number of line segments out from the centre

---

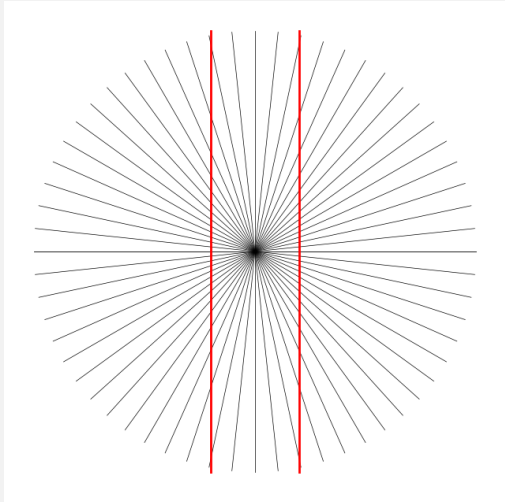
**Circle Boundary:** Defined by the length of the line (get it for free!)

---

**Two Parallel Lines:** Fixed distance apart, vertical orientation

---

# PATTERNS



- Each line segment
  - same length and width
  - equally spaced around the circle
  - Radiate out from the same starting point
    - (or are centered at that same point)
- The two vertical lines are parallel and same length / width, equal distance from the center point
- **Insight** – there is **symmetry** - the pattern is rotationally symmetric around the centre.
- **Insight** – Length of lines creates the circle!

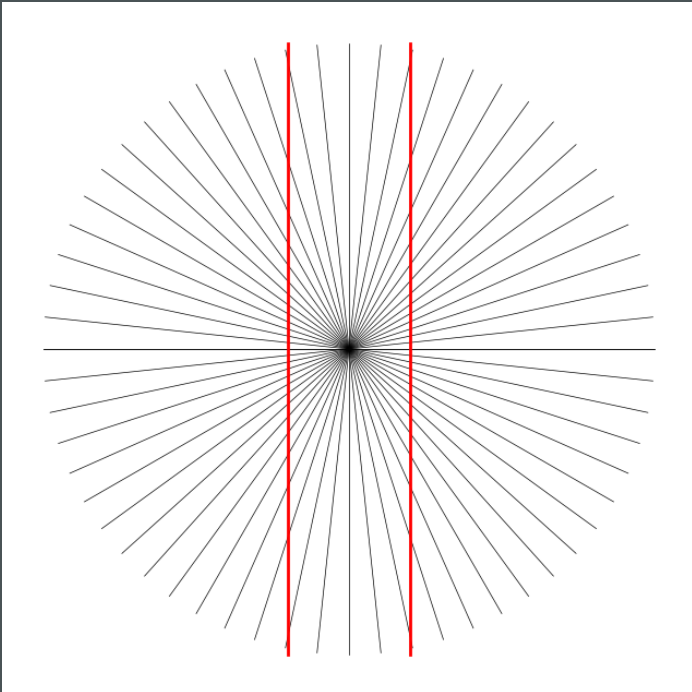
If we can draw one line segment at a specific angle from a specific point we can complete the whole illusion

# LOGIC

- How many “spokes”?
- Each spoke has an exact mirror
  - This could reduce computation and complexity
- Angles between each spoke should be equal
- Use our insight of symmetry:
  - Angles should divide evenly into 360 degrees – why?
  - Parallel lines are spaced equally on either side of the center

# DECOMPOSITION

Breaking the problem down into  
even smaller steps



---

**Center Point:** Define a centre point.

---

**Radial Lines:** Draw a line, rotate  $360/(\text{number of lines})$  and repeat

---

**Circle Boundary:** Defined by the x and y distance from the centre point for the end of the line

---

**Two Parallel Lines:** Fixed distance apart, vertical orientation

# ALGORITHM



**A set of step-by-step instructions**



**Uses Pseudo – code**



**Systematic approach**



**Leads to predictable outcome**

Algorithms are different than programming – a computer program will often **implement an algorithm**

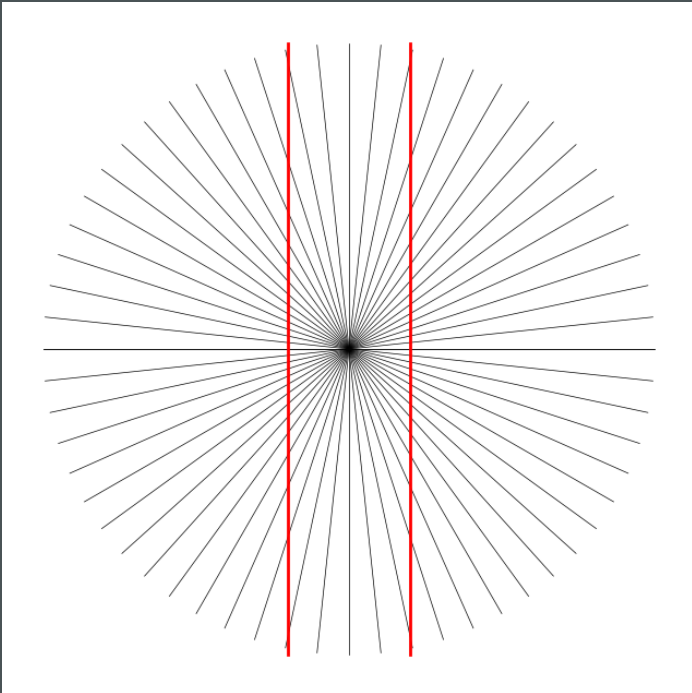
# PSEUDO CODE

- Human readable / informal
- Not a programming language (meaning that it cannot be interpreted by a machine directly)
- Outline the logic and structure using plain language and programming constructs
  - Steps combined into Blocks
  - Blocks may repeat
  - Blocks may only occur when particular facts are true (decision points)
- Helps with planning before we move to trying to translate it into a programming language



# KNOWN AND UNKNOWN

What facts do we know?



---

**Center Point:** Define a centre point. ✓

---

**Radial Lines:** Draw a line, rotate  $360/(\text{number of lines})$  and repeat ?

---

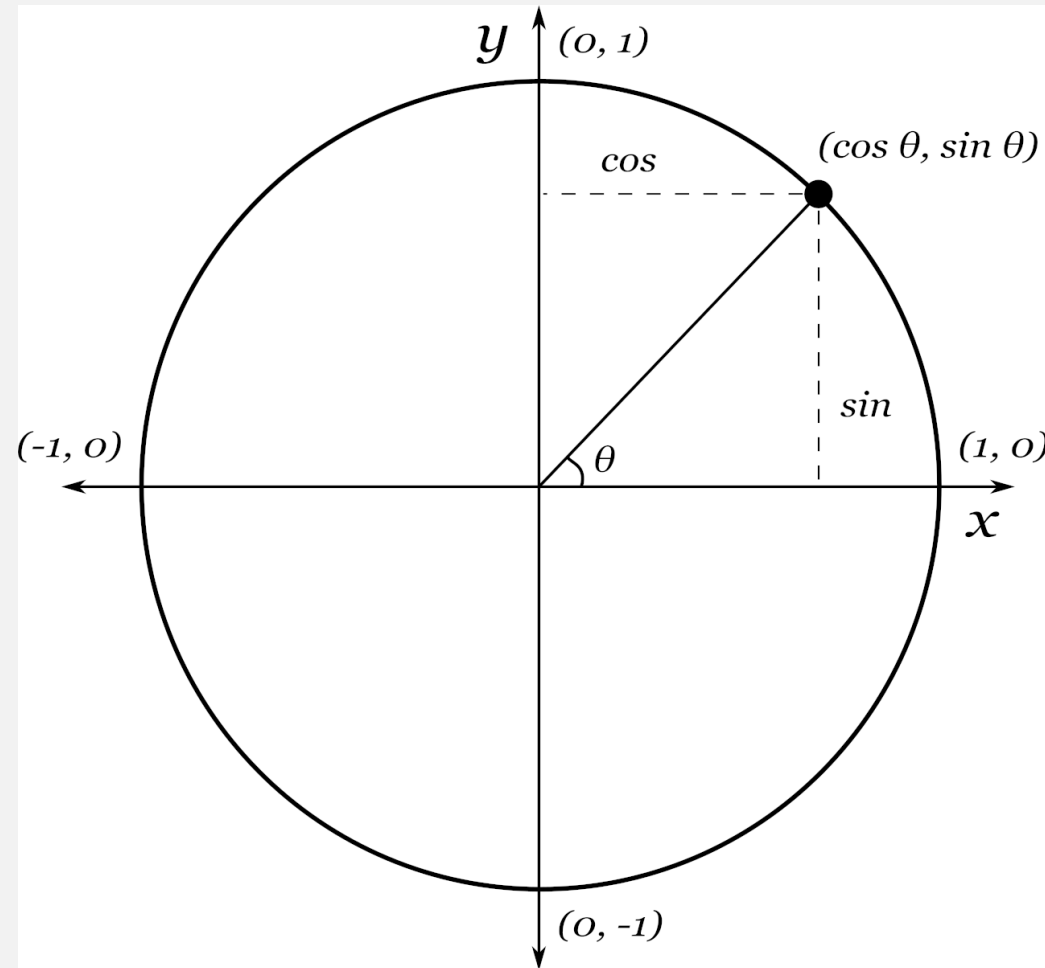
**Circle Boundary:** Defined by the x and y distance from the centre point for the end of the line ?

---

**Two Parallel Lines:** Fixed distance apart, vertical orientation ✓



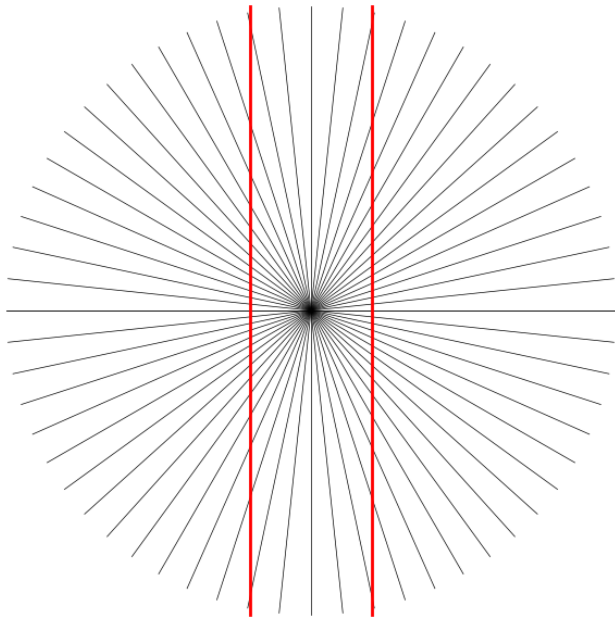
# UNIT CIRCLE





Class Discussion  
Solution on board

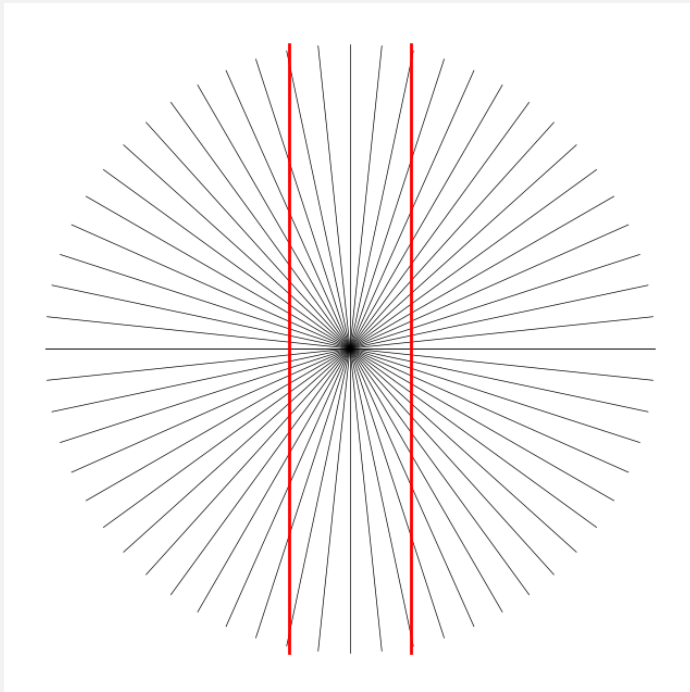
## PSEUDO CODE





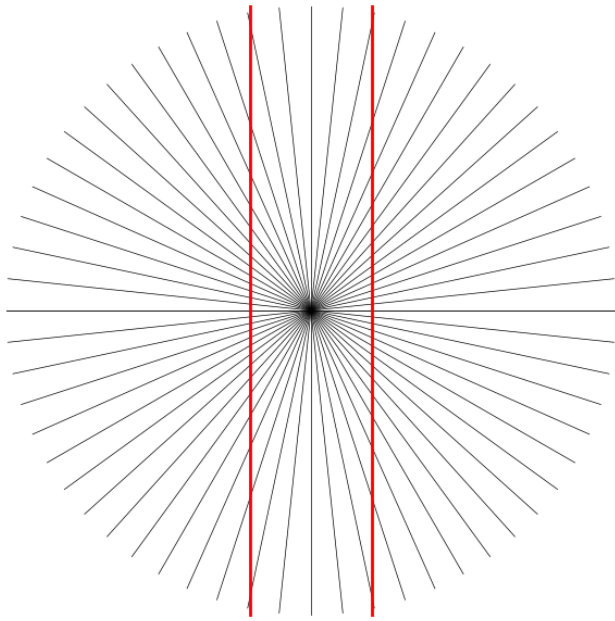
## PSEUDO CODE

- START
  - Set a length and a spacing and a number of spokes such that  $360/\text{spokes}$  is an even number (why? – we need a length and angle)
  - For each spoke:
    - Calculate the angle as prior angle +  $360/\text{spokes}$  degrees
    - Draw the line segment from 0,0 to  $\text{length} * \cos(\text{angle in radians})$ ,  $\text{length} * \sin(\text{angle in radians})$
  - Draw two parallel lines each spacing distance from (0,0) the parallel lines should be  $2 * \text{length}$  and centered at 0,0
- END



## ALTERNATIVE PSEUDO CODE

- START
  - Set a length and a spacing and a number of spokes such that  $360/\text{spokes}$  is an even number (why? – we need a length and angle)
  - For each spoke:
    - Calculate the angle as prior angle +  $360/\text{spokes}$  degrees
    - Draw a line segment from the left side of the circle to the right side of the circle so that I have half as many lines to draw.
  - Draw two parallel lines each spacing distance from (0,0) the parallel lines should be  $2 * \text{length}$  and centered at 0,0
- END



# PROGRAMMING

- Obviously, this is a little ahead of our Python learning
- \*\* Do not be alarmed you are not expected to be able to go from the pseudo code to this today

```
1 #This is a variant where we draw each individually spoke
2 # import some modules to help with math and plotting
3 import matplotlib.pyplot as plt
4 import numpy as np
5
6 # how many spokes do we want?
7 # spokes should divide evenly into 360
8 spokes = 60
9
10 # some setup for matplotlib
11 fig, ax = plt.subplots(figsize=(8, 8))
12 ax.set_aspect('equal')
13 ax.axis('off')
14
15 # angle between each spoke
16 # // does integer division
17 angle_delta = 360 // spokes # e.g., 60 -> 6 degrees per spoke
18
19 # Draw radiating spokes around the FULL 360 degrees
20 # We draw each spoke individually from the origin out to the rim
21 for angle in range(0, 360, angle_delta):
22     # endpoint of the spoke at this angle (half-line from the center)
23     x_end = 10 * np.cos(np.radians(angle))
24     y_end = 10 * np.sin(np.radians(angle))
25
26     # plot the spoke from the center (0,0) to the endpoint
27     ax.plot([0, x_end], [0, y_end],
28             color='black', linewidth=0.5)
29
30 # Draw the parallel lines (vertical red lines)
31 ax.plot([-2, -2], [10, -10], color='red', linewidth=2)
32 ax.plot([ 2,  2], [10, -10], color='red', linewidth=2)
33
34 plt.show()
```

# PROGRAMMING

```
8 #how many spokes do we want?
9 #spokes should divide evenly into 360
10 spokes = 60
11
12 #some setup for matplotlib lib
13 fig, ax = plt.subplots(figsize=(8, 8))
14 ax.set_aspect('equal')
15 ax.axis('off')
16
17 #angle between each spoke
18 #// does integer division
19 angle_delta = 360//spokes
20
21 # Draw radiating lines note the 180 degrees
22 for angle in range(0, 180, angle_delta):
23     #from -10 units to +10 units either side of center
24     x_start = -10 * np.cos(np.radians(angle))
25     y_start = -10 * np.sin(np.radians(angle))
26
27     x_end = 10 * np.cos(np.radians(angle))
28     y_end = 10 * np.sin(np.radians(angle))
29
30     #plot the line
31     ax.plot([x_start, x_end], [y_start, y_end],
32           color='black', linewidth=0.5)
33
34 # Draw the parallel lines
35 ax.plot([-2, -2], [10, -10], color='red', linewidth=2)
36 ax.plot([2, 2], [10, -10], color='red', linewidth=2)
37
38 plt.show()
```

List of angles but only covering half a circle

# VALIDATION

- Does the illusion visually appear when plotted?
- Are the spokes equally spaced?
- Are we missing any spokes (the last one)?
- Is the plot centered?



# TURTLE DRAW

A simple drawing library in python that allows us to control a “Turtle” that can leave a trail as we move it around the screen

Great for learning programming concepts

It will show up frequently in the course

Allows a visual understanding of our instructions (code)

Pseudo code and actual code look really similar

\* A program that prints hello world to the screen is a traditional first CS program

~~HELLO WORLD~~\* HELLO TURTLE LINE

```
1 #Hello Turtle
2
3 # Import the required module
4 from ColabTurtlePlus.Turtle import *
5 clearscreen()
6 forward(50)
```

A hash mark (#) indicates that the rest of the writing on this line is not code to be executed but rather a **comment**, often to help explain something or aid readability

function calls (a function is a group of code to accomplish something)

## WHAT CAN TURTLE DO (API)

- forward (units) – move turtle forward by units
- backward(units) – move turtle backward by units
- right(degrees) – rotate to the right by degrees
- left(degrees) – rotate to the left by degrees
- up() – pick the pen up (future movement will produce no trail)
- down() – put the pen down
- setx(x), sety(y), set the turtle's position

A selection of things available  
in the turtle library

## DRAW A SQUARE

- Side length 50 units

## DRAW A SQUARE

```
1 #Draw a Square
2
3 # Import the required module
4 from ColabTurtlePlus.Turtle import *
5
6 # Initialize the turtle screen
7 clearscreen()
8
9 #draw the 4 sides
10 forward(50)
11 right(90)
12
13 forward(50)
14 right(90)
15
16 forward(50)
17 right(90)
18
19 forward(50)
20 right(90)
```



# TURTLE CODE HERING ILLUSTRATION

- Old Pseudo Code with plotting:
- Given  $360 / \text{num of spokes angles}$
- For each angle
  - draw a line segment from 0,0 to  $\text{length} * \cos(\text{angle}), \text{length} * \sin(\text{angle})$
- Turtle Pseudo Code:
- Develop as class on board

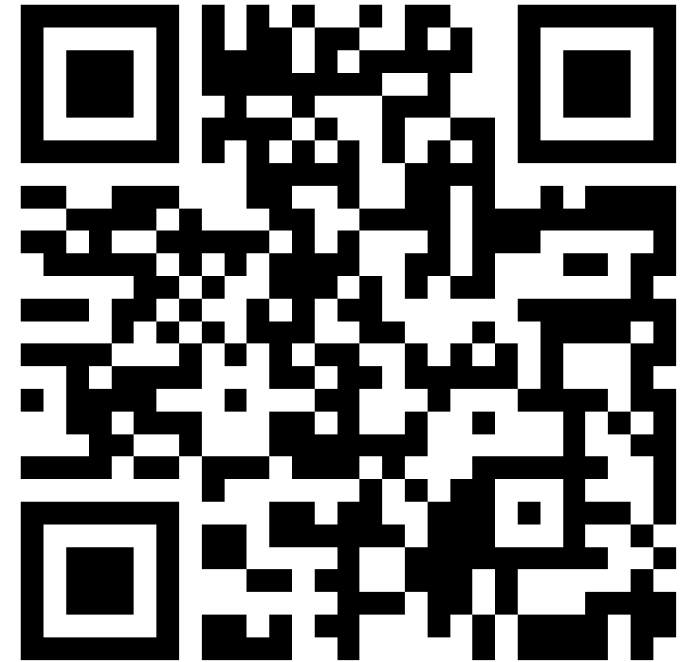
We already did the decomposition and pattern matching steps, DO NOT forget that

# TURTLE CODE PROGRAM

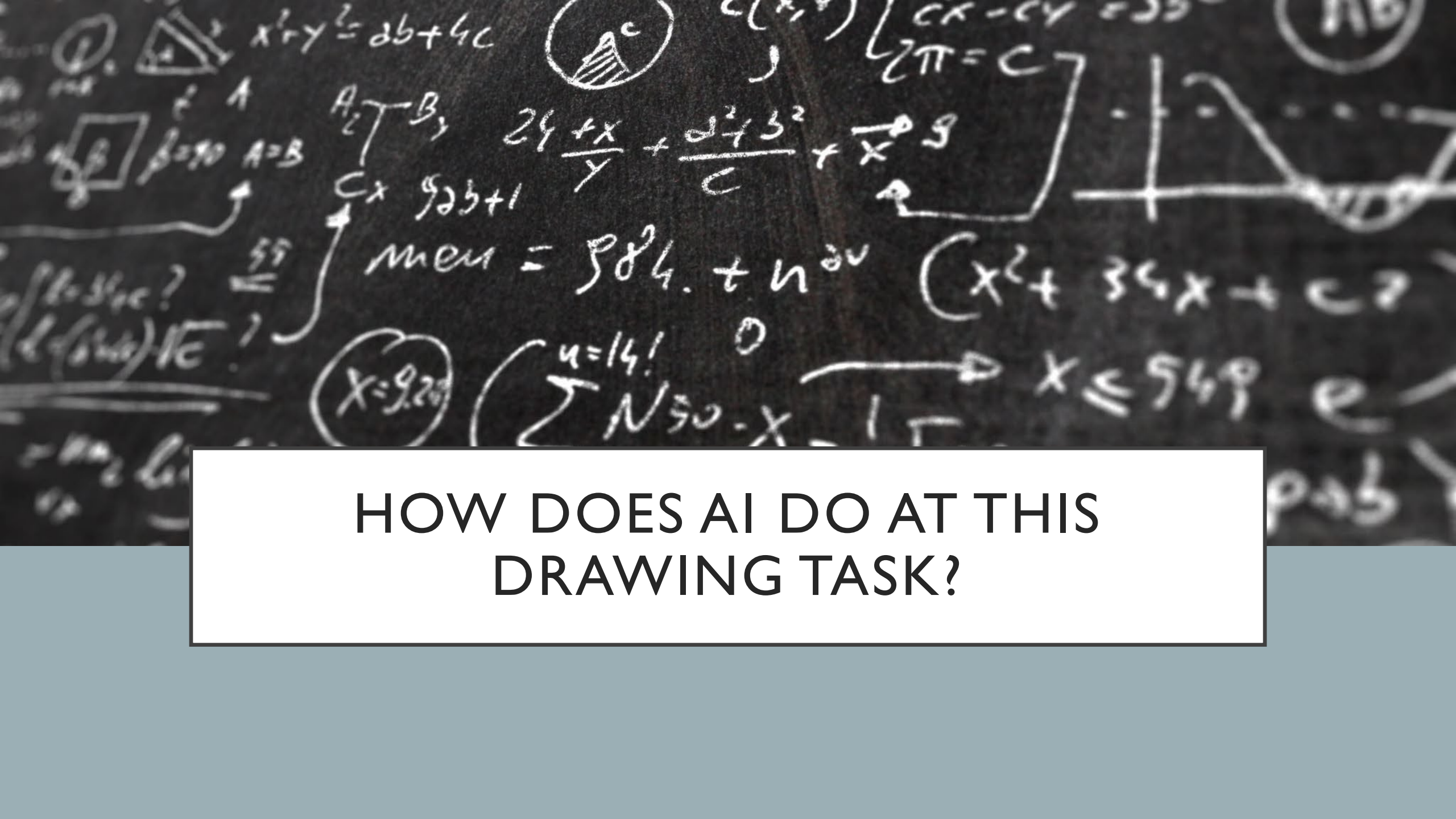
- Develop as class in colab
- Can we create a program with 4 spokes (N, S, E, W) and two parallel lines

# IN CLASS PARTICIPATION

[2026W CS1910 In-Class Participation 02 – Fill out form](#)





The background is a dark, textured surface resembling a chalkboard, filled with various handwritten mathematical expressions and diagrams in white. These include algebraic equations like  $x^2 + y^2 = 2b + 4c$ ,  $24 + \frac{x}{y} + \frac{2^2 + 3^2}{c} + \frac{1}{x} = 9$ , and  $men = 584 + n^{20}$ . There are also geometric diagrams, such as a circle with a shaded sector and a graph of a downward-opening parabola. Other visible text includes  $A_2 \perp B$ ,  $A=B$ ,  $\beta=90$ ,  $\sum N_{50-X-1}$ ,  $x=9.20$ ,  $n=14!$ ,  $x \leq 549$ , and  $C(x, y)$ .

HOW DOES AI DO AT THIS  
DRAWING TASK?

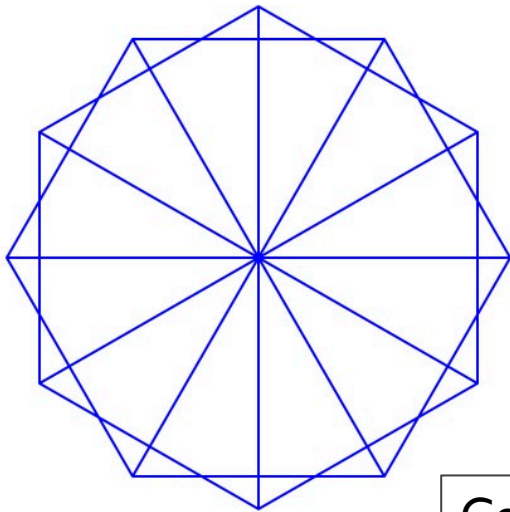
# AI DRAWING TASK

- What if we ask for code instructions?

A python plotting library

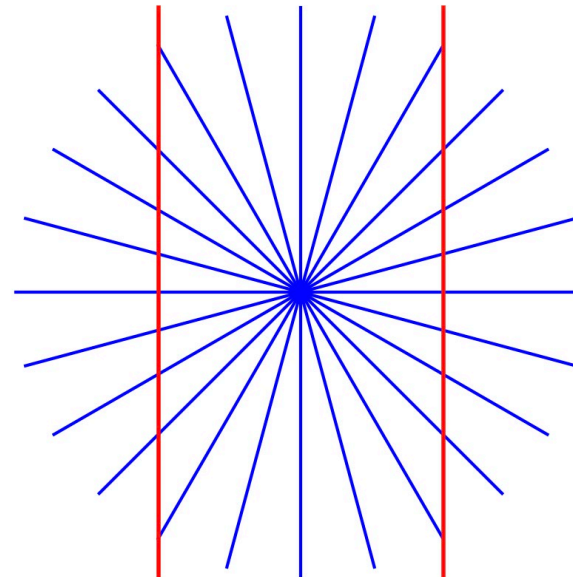
“Turtle draw pinwheel shape with two parallel lines on either side of the center”

“give me matplotlib code to draw a circle out of line segments as spokes and then add two parallel vertical lines on either side of the center of the circle”



Needs some refinement

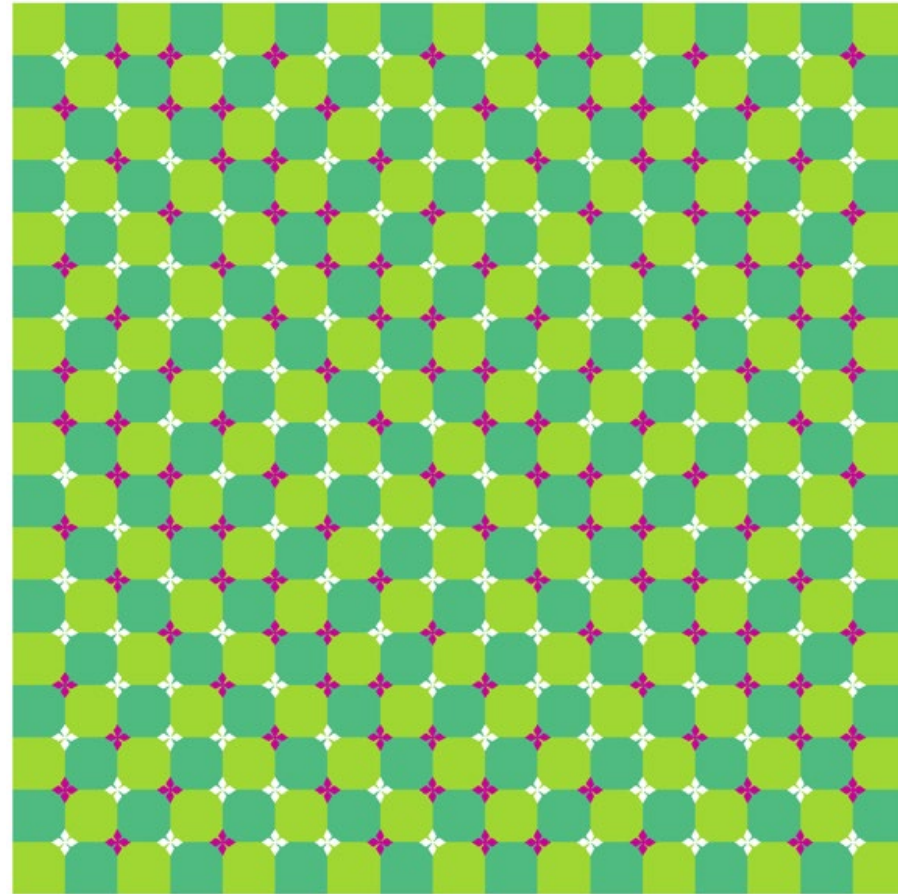
Could we have predicted this?



Pretty good

## TRY AT HOME

Use GenAI to recreate  
the green grid wavey  
illusion



Class Participation survey – how many text prompts do you think it will take you?

# USING GENAI

Under Tools -> Settings->AI Assistance  
Uncheck hide Gen AI features

This will produce the Gemini button at top right. Click that and proceed to enter prompts then cut and paste the code into a code block and run the code

Give instructions here and paste the results back into the notebook

**Turn off GenAI once you are done**

Note: it is challenging to unsee an answer

The screenshot shows a Jupyter Notebook interface. The top bar includes the Colab logo, the file name 'Untitled11.ipynb', and a menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. The 'Tools' menu is circled in red. Below the menu bar, there are tabs for 'Commands', '+ Code', '+ Text', and 'Run all'. The main area of the notebook displays a grid plot with alternating green and yellow squares, with black diamond shapes at the vertices. To the right of the notebook, a Gemini chat window is open. The chat window has a title bar 'Gemini X' and a search bar. The chat history shows a prompt: 'give matplotlib code to draw a green and greenyellow alternating grid with inverted diamond shapes at the vertices but only the interior vertices'. The Gemini response is a code block containing the following Python code:

```
import matplotlib.pyplot as plt
import numpy as np

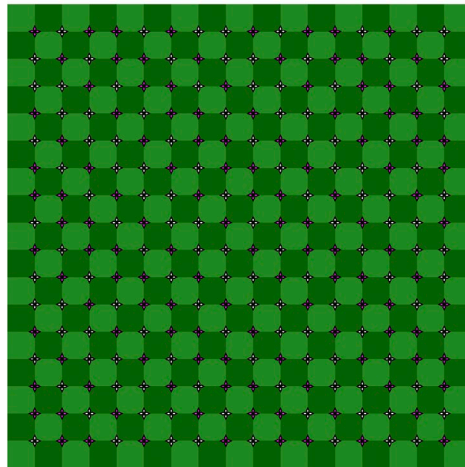
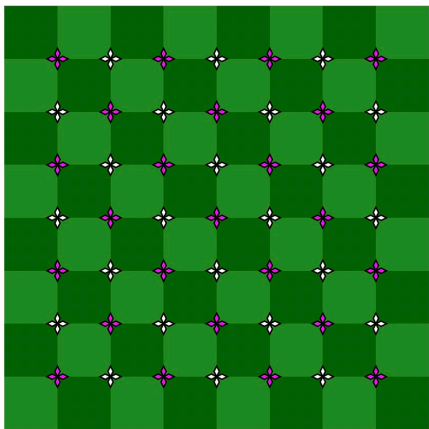
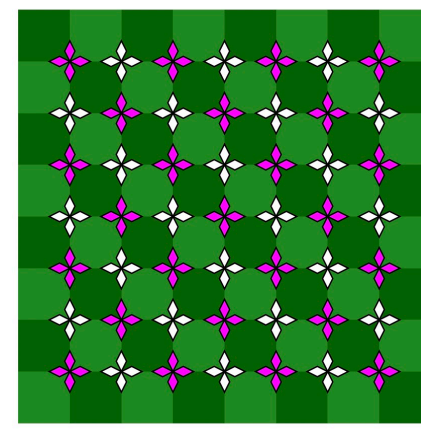
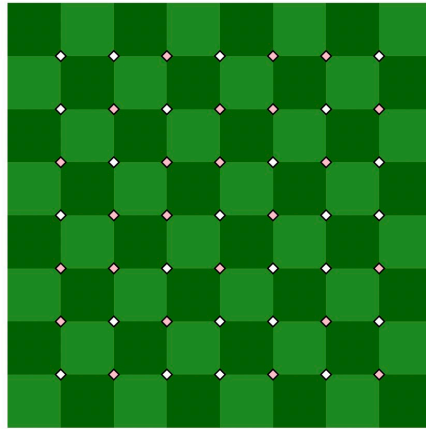
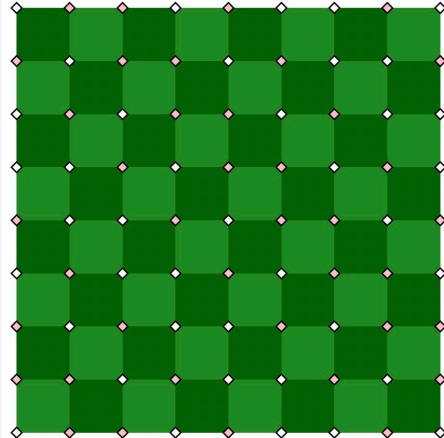
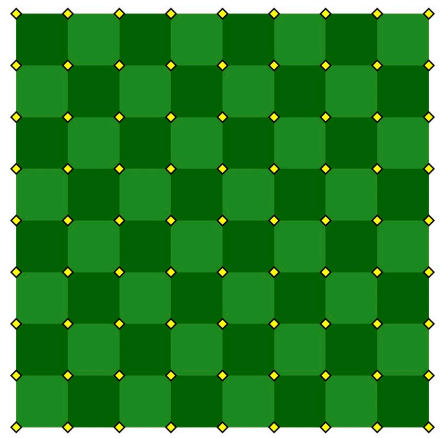
# Define grid parameters
grid_size = 10
cell_size = 1

# Create figure and axes
fig, ax = plt.subplots(figsize=(grid_size * cell_size, grid_size * cell_size))

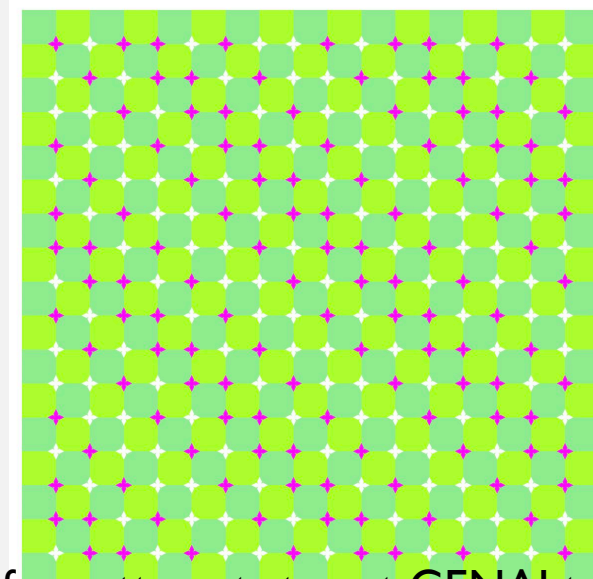
# Draw alternating grid
for i in range(grid_size):
```



# ITERATIVE PROCESS



...

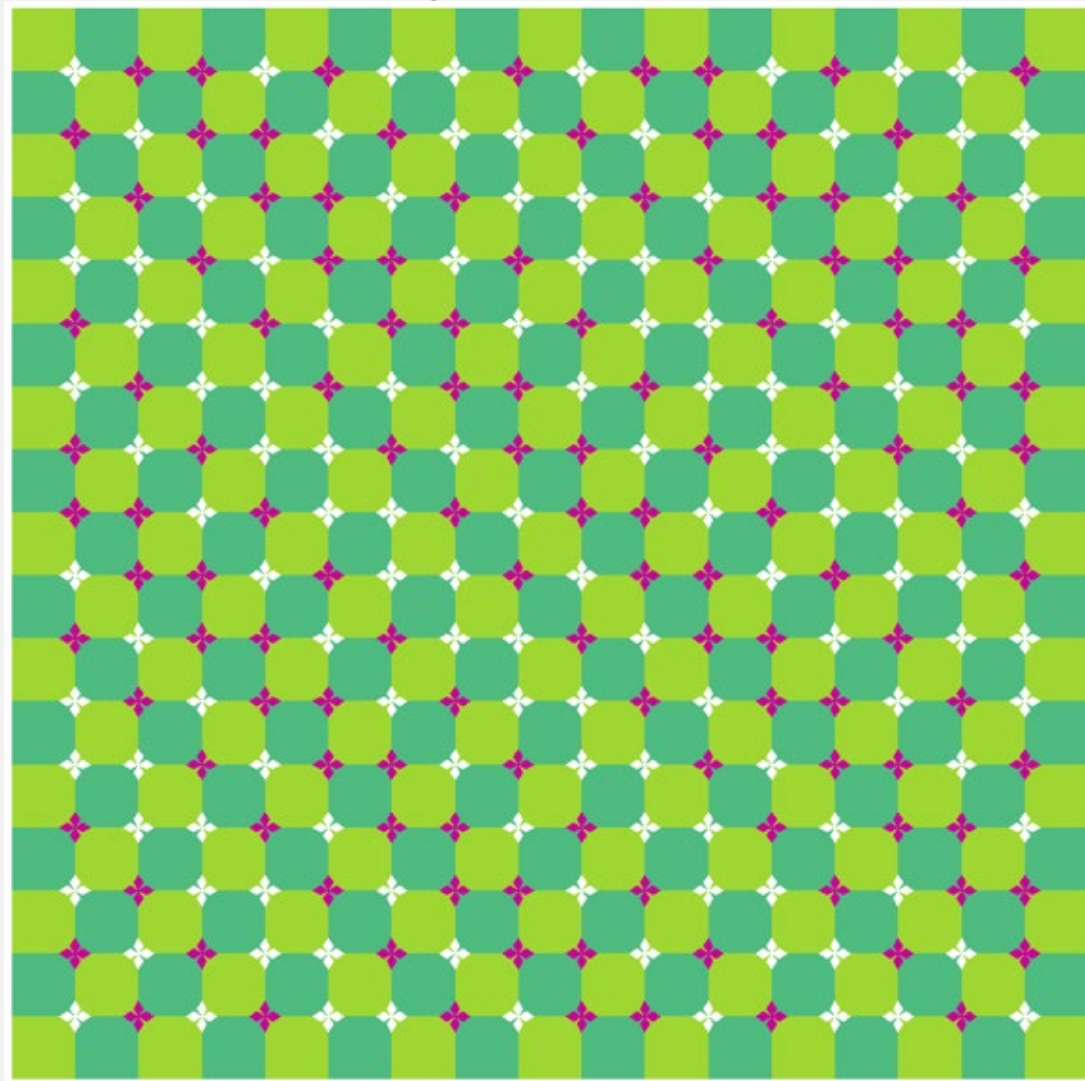


Think about some  
of our  
computational  
thinking skills

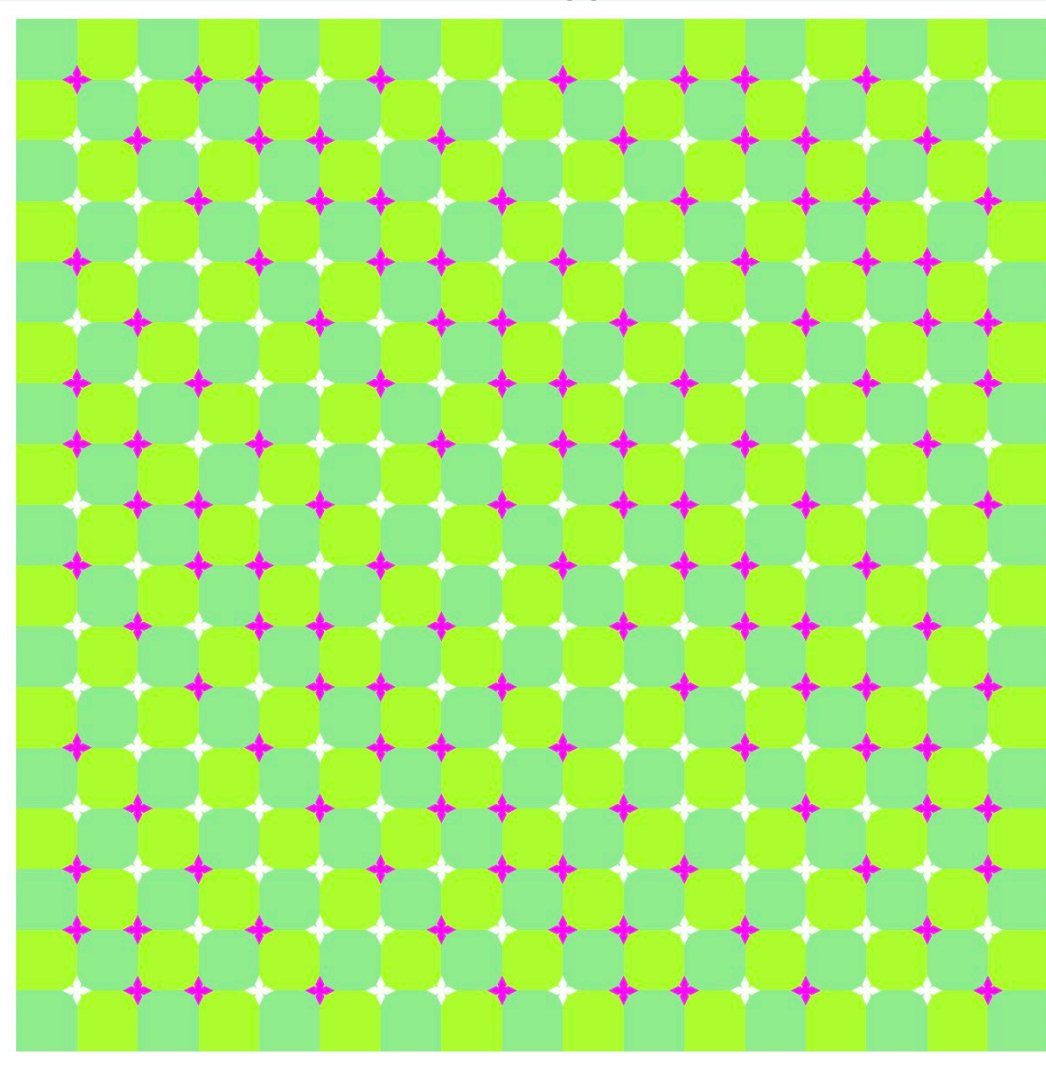
Decompose, logic,  
validation,  
patterns

A few of my attempts to get GENAI to create the illusion

Target

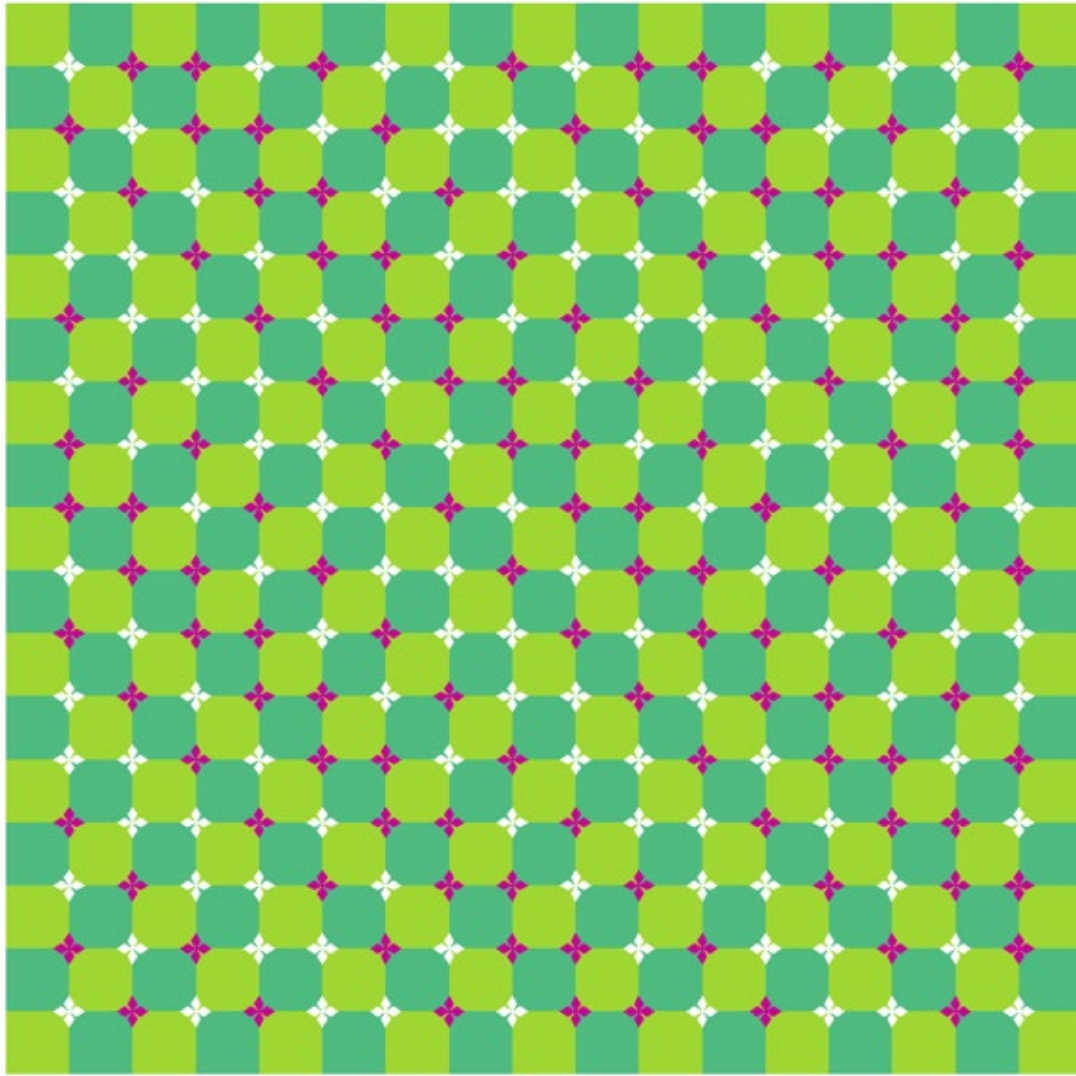


GENAI – iterative approach

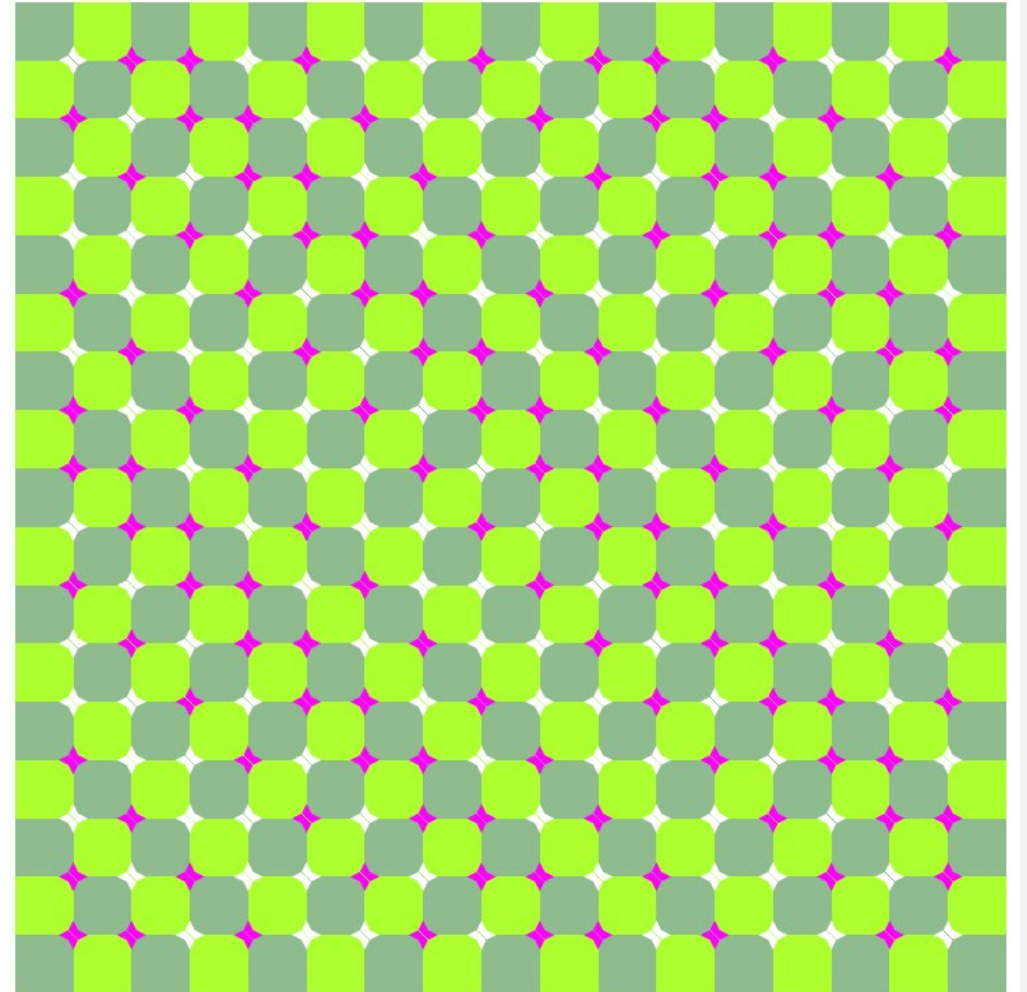




Target



Hand written code



\* Full disclosure the solution was originally coded then later prompted a GenAI to make a solution  
It is certainly one reason why the GenAI one looks like the coded one – the prompts to GenAI telling it to do things was a mimic of a lot of the analysis that we were done providing what is known as **context** for the AI the code

# TRY AT HOME

- Use Turtle Draw to create a triangle
- Use Turtle draw to create an X shape
- Use Turtle draw to create a block letter of your first initial in your name

Make sure GenAI  
is off, this is code  
you write

*#possibleexamcodingquestion*

- Jump in and modify the example code from today - make it do something different
  - Change the color in the primrose grid
  - Make it larger
  - Draw more spokes in the Hering Illustration
  - Draw horizontal parallel lines instead of vertical
  - Other ideas / tinkering

Tinker and/or be creative